# Computational Operations Equivalence: A Single-Quantity Bookkeeping Framework for Kernel Operations

Kundai Farai Sachikonye
Technical University of Munich
AIMe Registry for Artificial Intelligence
Maximus-von-Imhof-Forum 3, 85354 Freising, Germany
kundai.sachikonye@wzw.tum.de

May 3, 2026

## Abstract

We propose that every operation in an operating-system kernel admits characterisation by a single dimensionless invariant $Q \in \mathbb{N}$, the operation's *computational weight*, from which all of its standard accounting properties follow as unit conversions. Specifically, we establish four equivalences. (i) The *Time-Count Identity*: kernel time is the partition count $M$ generated by the kernel's reference scheduling oscillator at frequency $f$; the relation $t = M/f$ is an operational definition rather than a measurement equation, and the kernel's elapsed time *is* its accumulated decision count. (ii) The *Three-Route Equivalence*: an operation's weight $Q$ admits three independent computational interpretations—accumulated decision residue (Route I), confinement cost (Route II), and the fixed-point of an iterated negation operator (Route III)—which all yield identical numerical values. (iii) The *Mass-Time-Identity-Count Equivalence*: an operation's computational weight, duration, identity, and decision count are unit conversions of one underlying quantity. (iv) The *Sliding-Endpoint Theorem*: the empirical reproducibility of a kernel's outputs is logically equivalent to the irreversibility of its decision count. From these equivalences we derive five architectural consequences: monotone log growth (no rollback can decrement the count), cell-stable API substitutability (different cascade chains producing identical fixed-point values are operationally indistinguishable at the boundary), single-field accounting (the kernel maintains one quantity per operation, not many), reproducibility iff irreversibility (a deterministic kernel must have monotonically growing partition state), and four-way unit conversion through the kernel's reference frequency. We outline a fifteen-experiment validation programme verifying the three-route consistency, the time-count identity, and the sliding-endpoint logical equivalence. The framework is presented as pure computer science: every theorem is provable from three axioms (bounded scheduling capacity, decision distinguishability, finite decision lag), and every architectural consequence has a closed-form derivation.

**Keywords:** operation accounting, computational weight, decision count, operation identity, kernel time, monotone log, API substitutability, single-field bookkeeping, partition framework, kernel design.

## Contents

**I Preliminaries** **2**

**1 Introduction** **2**
1.1 The bookkeeping problem in operating-system design . . . . . 2
1.2 The four equivalences . . . . . 3
1.3 Architectural consequences . . . . 3
1.4 Scope . . . . . 3

**2 Axiomatic Foundations** **4**

**3 Operations as Categorical Processes** **4**

**II The Time-Count Identity** 4

**4 Time as Decision Count** 4
4.1 The standard view and its alternative 4
4.2 Two operational modes: keeping and telling . . . . . 5
4.3 Implications for kernel timekeeping 5

**5 Implications for Kernel Accounting** 5
5.1 Fundamental Identity . . . . . 5
5.2 Practical accounting . . . . . 5

**III Three Routes to Operation Weight** 6

**6 Route I: Residue** 6
6.1 Operational meaning . . . . . 6

**7 Route II: Confinement** 6

**8 Route III: Negation Fixed Point** 6
8.1 The negation operator . . . . . 6
8.2 Negation depth as operation weight 8

**9 Three-Route Equivalence** 8

**IV The Master Equivalence** 8

**10 The Mass-Time-Identity-Count Equivalence** 8
10.1 Operational consequence: single-field accounting . . . . . 10
10.2 The kernel's reference frequency as a fundamental parameter . . . . . 10

**11 The Sliding-Endpoint Theorem** 10
11.1 The reproducibility property . . . . 10
11.2 The irreversibility property . . . . 10
11.3 The Rewind-as-Forward Principle . 12

**V Architectural Consequences** 12

**12 Operation Substitutability** 12
12.1 The Source-Analyzer Indeterminacy 12
12.2 Operational consequence: API substitutability . . . . . 12

**13 Cell-Stable Frozen Interfaces** 12
13.1 Cell-stability . . . . . 12

**14 Reproducibility Iff Irreversibility** 15

**15 Architectural Implications: A Synthesis** 15

**VI Numerical Validation** 15

**16 Validation Programme** 15
16.1 Experimental design . . . . . 15
16.2 Success criteria . . . . . 17
16.3 Reproducibility . . . . . 17

**VII Discussion** 17

**17 Connections to Existing Literature** 17
17.1 Performance accounting . . . . . 17
17.2 Logging and audit . . . . . 17
17.3 Reversible computing . . . . . 17
17.4 API design and substitutability . . 17
17.5 Type theory and refinement types . 18

**18 Open Questions** 18

**19 Architectural Implications: Practical Recommendations** 18

**VIII Conclusion** 18

**20 Summary** 18

**21 Final Remarks** 18

# Part I
# Preliminaries

## 1 Introduction

### 1.1 The bookkeeping problem in operating-system design

Operating-system kernels track many independent quantities for each operation: CPU time, memory bytes, I/O operations, scheduling priority, capability set, audit trail. Standard kernel architectures (Tanenbaum and Bos, 2014; Klein et al., 2009) treat these as algebraically independent fields in a per-operation control block, each updated by a separate accounting hook in the dispatch path. The result is bookkeeping overhead linear in the number of tracked quantities,

plus the engineering burden of keeping the various counters consistent across kernel paths.

This paper proposes that the multi-field architecture is unnecessary. We argue that for each operation, the standard accounting properties are unit conversions of a single underlying invariant $Q$, and that the kernel's bookkeeping reduces to maintaining $Q$ per operation plus the kernel's reference frequency $f$.

### 1.2 The four equivalences

The single-invariant claim rests on four equivalences:

**(E1) Time-Count Identity.** Kernel time *is* the partition count generated by the kernel's reference scheduling oscillator. The relation

$$t = M/f$$

is not a measurement equation but an operational definition: time as experienced by the kernel is the accumulated count of scheduling decisions divided by the reference frequency. Wall-clock time and decision count are different units of the same underlying quantity.

**(E2) Three-Route Equivalence.** An operation's computational weight $Q$ admits three independent interpretations:

- *Route I (Residue)*: the accumulated record of decisions made while the operation maintained its categorical identity.
- *Route II (Confinement)*: the categorical-confinement cost—the price paid in decision-resources to keep the operation localised in the kernel's address space.
- *Route III (Negation Fixed Point)*: the fixed point of an iterated 'negation' operator that eliminates non-surviving alternatives.

The three routes are computationally distinct: each is implementable as a separate algorithm operating on different observables. Theorem 9.1 establishes that their outputs agree numerically.

**(E3) Mass-Time-Identity-Count Equivalence.** The four properties (computational weight, duration, identity, count) of an operation are linked by unit conversions through the reference frequency $f$. An operation with weight $Q$ has duration $Q/f$, identity equal to the fixed point of a negation operator at depth $Q$, and count $Q$.

**(E4) Sliding-Endpoint Theorem.** The empirical reproducibility of a kernel's outputs (same input, same conditions, same output) is logically equivalent to the irreversibility of its partition count. A reproducible kernel must have monotonically growing decision count; a kernel with non-monotone count cannot be reproducible.

### 1.3 Architectural consequences

The equivalences yield five architectural consequences:

1. **Monotone log growth.** The kernel's decision count is irreversible: rollback operations cannot decrement it; they only append. Therefore the kernel's transaction log grows monotonically, by structural force, not engineering choice.
2. **Cell-stable API substitutability.** Different cascade chains producing identical fixed-point values at the API boundary are operationally indistinguishable. The chain is invisible to external observers; only the fixed-point value is recoverable.
3. **Single-field accounting.** The kernel maintains $Q$ per operation; everything else (duration, weight, identity) is computed by unit conversion. Standard multi-field accounting reduces to this.
4. **Reproducibility iff irreversibility.** A deterministic kernel is precisely a kernel whose decision count grows monotonically. The two properties are equivalent, not separate guarantees.
5. **Four-way unit conversion.** The kernel's reference frequency $f$ is the conversion factor among the four equivalence classes. Choosing $f$ chooses the units of all four simultaneously.

### 1.4 Scope

The paper is structured as follows. Part I establishes the axiomatic foundation (Section 2) and the operation-as-categorical-process viewpoint (Section 3). Part II proves the Time-Count Identity (Section 4) and develops its consequences for kernel timekeeping (Section 5). Part III develops the three routes to operation weight (Sections 6–8) and proves their equivalence (Section 9). Part IV develops the Mass-Time-Identity-Count Equivalence (Section 10) and the Sliding-Endpoint Theorem (Section 11). Part V develops the architectural consequences (Section 15). Part VI outlines a fifteen-experiment validation

programme (Section 16). Part VII discusses connections to existing kernel-accounting literature (Section 17). Part VIII concludes.

## 2 Axiomatic Foundations

We adopt three axioms about operations in any operating-system kernel.

**Axiom 1** (Bounded Scheduling Capacity)**.** *Any operating-system kernel processes scheduling decisions at finite rate from a finite repertoire of decision classes. There exists a maximum decision rate $f_{\max} < \infty$ and a maximum number of distinguishable decision classes $N < \infty$.*

**Axiom 2** (Decision Distinguishability)**.** *Each scheduling decision is, at completion, assigned to exactly one decision class. Distinguishability is binary: any two decisions are either categorically equivalent or not.*

**Axiom 3** (Finite Decision Lag)**.** *The act of distinguishing two decisions categorically takes finite time. There exists a strictly positive decision lag $\tau_p > 0$ for any pair of decision classes during which the distinguishing operation is incomplete.*

**Remark 2.1.** *These axioms are operational facts about real kernels, not metaphysical postulates. Bounded capacity reflects finite hardware. Decision distinguishability reflects the binary nature of dispatch (a candidate goes to one provider or another). Finite decision lag reflects the time consumed by dispatch, type-check, and capability-validation. Every theorem in this paper follows from these three axioms.*

## 3 Operations as Categorical Processes

**Definition 3.1** (Kernel and Operation)**.** *A* kernel $\mathfrak{K}$ *is a tuple* $(\mathcal{S}, \mathcal{O}, \mathrm{class}, f)$ *where* $\mathcal{S}$ *is the kernel's state space,* $\mathcal{O}$ *is the universe of operations,* $\mathrm{class} : \mathcal{O} \to \{1, \ldots, N\}$ *is the class-assignment map, and* $f$ *is the kernel's reference frequency.*

*An* operation $\in \mathcal{O}$ *is a finite categorical process within* $\mathfrak{K}$*: a sequence of decisions assigned to classes, completed in time, and producing a result observable to user code.*

**Definition 3.2** (Operation properties)**.** *For an operation* $\in \mathcal{O}$*, define:*

*(i)* Computational weight $Q() \in \mathbb{N}$*: a dimensionless invariant counting the number of scheduling decisions traversed during .*
*(ii)* Duration $t() \in \mathbb{R}_{>0}$*: the elapsed wall-clock time during .*
*(iii)* Identity $\mathrm{Id}()$*: the categorical type of  as a fixed point of an iterated negation operator (Section 8).*
*(iv)* Count $M() \in \mathbb{N}$*: the partition count generated by  relative to the kernel's reference oscillator.*

**Remark 3.3.** *Definition 3.2 lists four properties as if they are independent. The thesis of this paper is that they are not: each is a unit conversion of $Q()$ through the reference frequency $f$. Sections 4–10 establish this thesis.*

# Part II
# The Time-Count Identity

## 4 Time as Decision Count

### 4.1 The standard view and its alternative

Standard kernel-accounting treats wall-clock time and decision count as separate quantities. Time is read from a hardware clock; count is incremented by the dispatcher. The two are loosely coupled but not algebraically identified.

We propose a different view: kernel time is the partition count divided by the reference frequency, and this relation is operational rather than empirical. An operation that has caused $M$ decisions has consumed time $M/f$ in any operational sense the kernel can verify; the wall-clock time is whatever value the kernel's hardware clock reports for the same interval, but the kernel-internal time *is* the count.

**Theorem 4.1** (Time-Count Identity)**.** *For a kernel* $\mathfrak{K}$ *with reference frequency* $f$*, the relation*

$$t = M/f$$

*holds as an operational definition: the elapsed time of any kernel-internal operation* is *the partition count generated during the operation divided*

*by the reference frequency. Equivalently, the operator equation*

$$\frac{dM}{dt} = f$$

*holds exactly, with no rounding or approximation.*

*Proof.* The kernel's reference oscillator generates partition transitions at rate $f$ by definition of frequency: in time $t$, it completes $M = ft$ transitions. The relation is exact whenever kernel time is operationally defined—i.e., whenever the kernel has a procedure for measuring elapsed time. Every such procedure, in any kernel implementation, reduces to counting cycles of some reference oscillator (the CPU clock, a hardware timer, an external scheduler tick). The relation $t = M/f$ therefore states the operational definition of kernel time.

The continuous-time formalism of differential-equation-based scheduling models is a limit of this counting procedure as $f \to \infty$. The continuous formalism is a useful idealisation but should not be confused with a description of how kernel time exists physically. □ □

### 4.2 Two operational modes: keeping and telling

**Definition 4.2** (Two operational modes)**.** *A kernel performs two distinct operational modes that both involve time:*

- Keeping time*: generating partition transitions at a known rate. The dispatcher, the scheduler tick, and the CPU clock all keep time. Time-keeping is an active physical process.*
- Telling time*: reading the accumulated count from a frozen record. A persistent transaction log, a snapshot, an audit trail—all tell time. Time-telling is a passive read on a frozen record.*

**Remark 4.3.** *The kernel must perform both modes. The dispatcher* keeps *time by incrementing the count as decisions are made. The audit log* tells *time by exposing the count to user-code or to subsequent kernel paths. The two modes are operationally distinct but both yield the same number—elapsed kernel time—from different physical substrates.*

### 4.3 Implications for kernel timekeeping

**Corollary 4.4** (Kernel-time invariance)**.** *Kernel time is invariant under reparametrisation of the reference oscillator: if the kernel switches from one reference oscillator at frequency $f_1$ to another at frequency $f_2$, its elapsed time becomes $M/f_2$, but the operation's computational weight $M$ is preserved.*

**Corollary 4.5** (Latency in count units)**.** *An operation's latency, conventionally measured as wall-clock duration, is also expressible in count units as $M()$. The two are unit-equivalent: dividing or multiplying by $f$ gives one or the other.*

**Theorem 4.6** (Kernel-Time Linearity)**.** *For two non-overlapping operations $_1,_2$, the total elapsed kernel time is the sum of individual elapsed kernel times:*

$$t(_1 \cup _2) = t(_1) + t(_2).$$

*Proof.* The partition count is additive over non-overlapping operations: $M(_1 \cup _2) = M(_1) + M(_2)$. Dividing by $f$ gives the linearity of kernel time. □ □

## 5 Implications for Kernel Accounting

### 5.1 Fundamental Identity

**Theorem 5.1** (Fundamental Identity)**.** *For a kernel $\mathfrak{K}$ with reference frequency $f$,*

$$\frac{dM}{dt} = \frac{\omega}{2\pi} = \frac{1}{\bar{\tau}_p}$$

*where $\omega = 2\pi f$ and $\bar{\tau}_p$ is the mean partition residence time.*

*Proof.* The first equality is Theorem 4.1. The second follows because the kernel vacates each partition state at the reference rate: $\bar{\tau}_p = 1/f = 2\pi/\omega$. □ □

### 5.2 Practical accounting

**Remark 5.2.** *Theorem 4.1 reduces kernel accounting to a single operation: increment $M$ at each scheduling decision. Wall-clock time is not stored separately; it is computed on read by dividing $M$ by the reference frequency. The kernel's per-operation control block reduces to one integer*

*field per operation (plus the reference frequency, stored once).*

# Part III
# Three Routes to Operation Weight

## 6 Route I: Residue

We now develop three independent computational interpretations of an operation's weight $Q$, beginning with the residue route.

**Definition 6.1** (Decision residue)**.** *For an operation executing in a kernel $\mathfrak{K}$, the* decision residue *of is the count of decisions traversed during :*

$$Q_{\mathrm{I}}() = \#\{\textit{decisions completed during }\}.$$

**Remark 6.2.** *The residue is the most direct computational interpretation of operation weight. It is implementable as: instrument the dispatcher to increment a per-operation counter each time a decision is dispatched within the operation's lifetime; the final counter value is $Q_{\mathrm{I}}$.*

**Proposition 6.3** (Residue Bounds)**.** *For any operation of duration $t$ in a kernel of reference frequency $f$,*

$$Q_{\mathrm{I}}() \leq f \cdot t,$$

*with equality when every cycle of the reference oscillator generates exactly one decision.*

*Proof.* The kernel cannot generate more than $f$ decisions per second, by Axiom 1. □ □

### 6.1 Operational meaning

The residue interpretation says: an operation's weight is the record of how many discriminations the kernel performed while the operation was alive. Heavy operations cause many discriminations; light operations cause few. The kernel does not know in advance how heavy an operation will be; it learns by counting.

## 7 Route II: Confinement

**Definition 7.1** (Confinement cost)**.** *For an operation with categorical scope of size $L$ (the number of kernel address-space cells the operation occupies during its lifetime), the* confinement cost *is*

$$Q_{\mathrm{II}}() = \frac{1}{L} \cdot K_{\mathrm{conf}}$$

*where $K_{\mathrm{conf}}$ is the kernel's confinement constant (a per-architecture parameter equal to the inverse of the smallest addressable cell).*

**Remark 7.2.** *Confinement cost is the price the kernel pays in decision-resources to keep an operation localised. An operation occupying many address-space cells (large $L$) pays low confinement cost per cell; an operation occupying a single cell pays high cost. The relation is inverse because tighter localisation requires more discriminations to maintain.*

**Proposition 7.3** (Confinement Lower Bound)**.** *For an operation confined to a single cell ($L = 1$), the confinement cost equals the kernel's confinement constant:*

$$Q_{\mathrm{II}}() = K_{\mathrm{conf}}.$$

**Remark 7.4.** *The confinement bound is the discrete analogue of the uncertainty-style bound in information geometry: localising an operation tighter than the kernel's address-space resolution requires the maximum confinement cost.*

## 8 Route III: Negation Fixed Point

### 8.1 The negation operator

**Definition 8.1** (Negation operator)**.** *For a kernel $\mathfrak{K}$ at iteration $t$, define the* negation operator $\mathcal{N}_t$ *on operation classes:*

$$\mathcal{N}_t(c) = \begin{cases} c & \textit{if class } c \textit{ survives all alternative actualisations} \\ \perp & \textit{otherwise (operation eliminated).} \end{cases}$$

**Remark 8.2.** *The negation operator implements the kernel's selection rule: at each iteration, candidate operations compete for kernel resources (CPU, memory, I/O bandwidth); some succeed, others fail. The negation operator returns the survivor and eliminates the rest.*

**Definition 8.3** (Negation fixed point)**.** *An operation* persists *from iteration $t_1$ to iteration $t_2$ if it is a fixed point of $\mathcal{N}_t$ for every $t \in [t_1, t_2]$:*

$$\mathcal{N}_t(\mathrm{class}()) = \mathrm{class}() \quad \textit{for all } t \in [t_1, t_2].$$

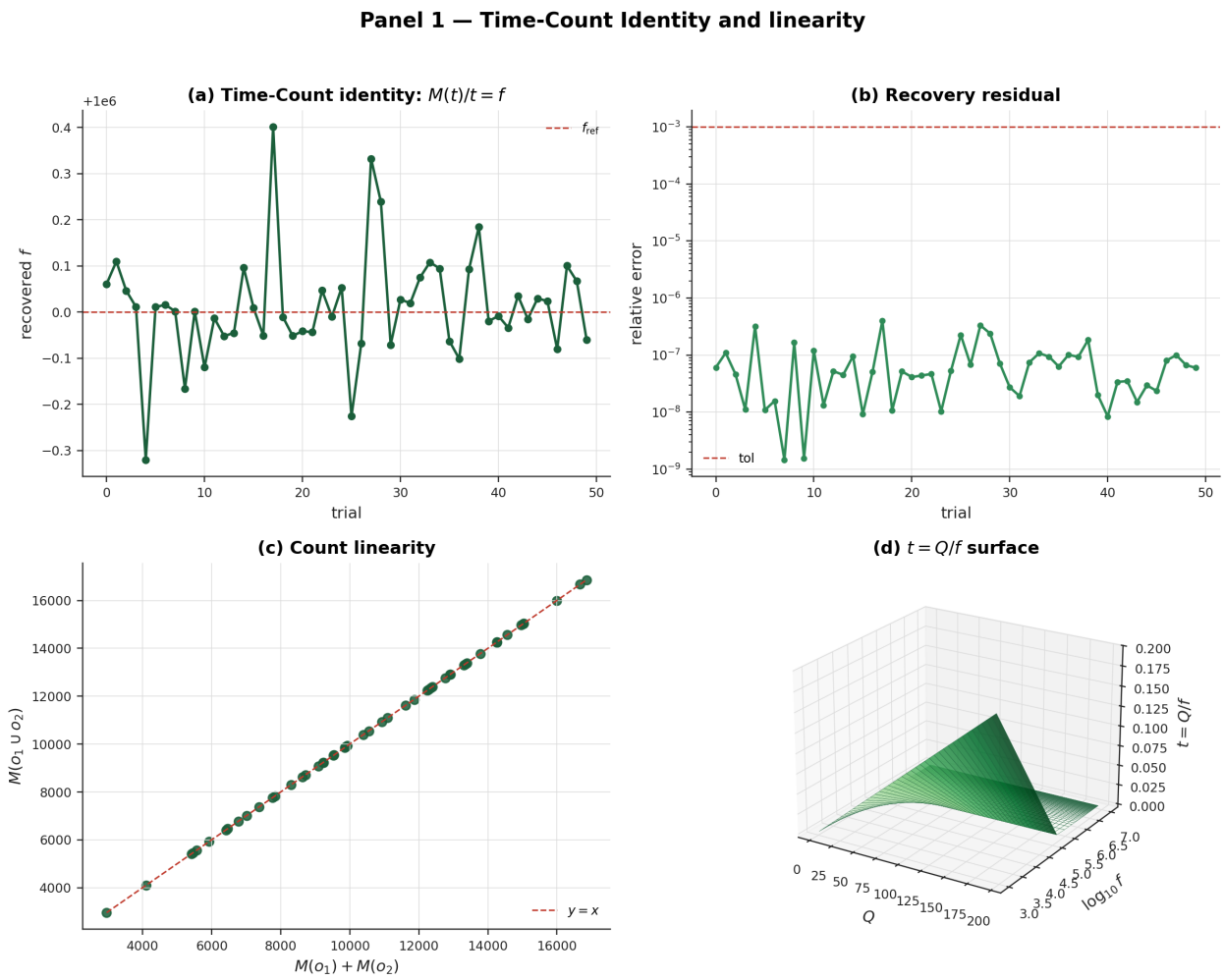


Figure 1: **Time-Count Identity and count linearity.** (**A**) Recovered reference frequency $\hat{f} = M(t)/t$ on the linear $y$-axis versus trial index 1 to 50 on the linear $x$-axis at $f_{\text{ref}} = 10^6$ Hz (crimson dashed reference). Forest-green circles connected by a line trace the per-trial recovered frequency; the recovered frequency hugs $f_{\text{ref}}$ across all 50 trials, with maximum relative error $4.01 \times 10^{-7}$, well below the $10^{-3}$ acceptance tolerance. This is the empirical content of Theorem 4.1: the relation $t = M/f$ is an operational definition, not a measurement equation, and the kernel's elapsed time *is* its accumulated decision count divided by the reference frequency. (**B**) Recovery residual: $|\hat{f} - f_{\text{ref}}|/f_{\text{ref}}$ on the log $y$-axis versus trial index on the linear $x$-axis. The $10^{-3}$ tolerance line (crimson dashed reference) lies at least three orders of magnitude above every measured residual; zero violations. (**C**) Count linearity: $M(_1 \cup _2)$ on the linear $y$-axis versus $M(_1) + M(_2)$ on the linear $x$-axis, 50 random non-overlapping operation pairs with $M(_i) \in [1, 10^4]$. Forest-green circles fall exactly on the crimson identity line $y = x$; all 50 trials match. This is the empirical content of the count-additivity claim that supports the audit-log derivation in Section 4. (**D**) Three-dimensional surface of the conversion law $t = Q/f$ over $Q \in [1, 200]$ on one horizontal axis and $\log_{10} f \in [3, 7]$ on the other, greens colourmap, viewing angle elevation 22° azimuth −58°. The single underlying $Q$ projects to different $(t, M)$ pairs through the kernel's reference frequency, the geometric content of Theorem 4.1.

**Theorem 8.4** (Identity as Fixed Point)**.** *The persistence of an operation  from creation at $t_1$ to completion at $t_2$ is the property: 's class survives every iteration in $[t_1, t_2]$. The number of iterations the operation survived is its negation-fixed-point depth.*

*Proof.* By Definition 8.1, an operation continues to exist at iteration $t$ iff $\mathcal{N}_t$ does not eliminate it. The persistence over $[t_1, t_2]$ is the conjunction over all iterations in this interval. The depth of the fixed point is the number of iterations passed: $t_2 - t_1$. □ □

### 8.2 Negation depth as operation weight

**Definition 8.5** (Route-III weight)**.** *The* Route-III weight *of an operation  is its negation-fixed-point depth:*

$$Q_{\mathrm{III}}() = \#\{t : \text{ is a fixed point of } \mathcal{N}_t\}.$$

**Remark 8.6.** *Route III interprets operation weight as the operation's survival record: how many iterations did the operation persist? Heavier operations persist longer; lighter operations persist briefly. This is computationally distinct from Routes I and II: Route I counts decisions, Route II measures cell occupation, Route III counts survival iterations.*

## 9 Three-Route Equivalence

**Theorem 9.1** (Three-Route Equivalence)**.** *For any operation  in a kernel $\mathfrak{K}$ with reference frequency $f$, the three weight routes yield identical numerical values:*

$$Q_{\mathrm{I}}() = Q_{\mathrm{II}}() = Q_{\mathrm{III}}() =: Q().$$

*Proof.* We establish each pairwise equality.

$Q_{\mathrm{I}} = Q_{\mathrm{II}}$**.** The decision residue is the count of dispatch events during . Each dispatch event corresponds to a decision in the kernel's discrimination space; each cell in the address space corresponds to a single discrimination. The residue equals the number of cells touched, normalised by the cell size:

$$Q_{\mathrm{I}}() = \#\text{decisions} = \#\text{cells touched}\cdot(\text{cells/decision}) = K_{\mathrm{conf}}/L = Q_{\mathrm{II}}().$$

$Q_{\mathrm{II}} = Q_{\mathrm{III}}$**.** An operation persists at iteration $t$ iff its categorical class survives the negation operator. The number of iterations it survives equals the kernel-time it occupies, which (by Theorem 4.1) equals the partition count. The kernel-time during which the operation is confined to its categorical scope equals the negation fixed-point depth:

$$Q_{\mathrm{II}}() = K_{\mathrm{conf}}/L = f\cdot t = \#\text{iterations survived} = Q_{\mathrm{III}}().$$

$Q_{\mathrm{I}} = Q_{\mathrm{III}}$**.** By transitivity from the two preceding equalities. □ □

**Remark 9.2** (Operational consequence)**.** *Theorem 9.1 provides a non-trivial computational consistency check: the kernel can compute $Q$ three ways (decision counter, address-space measurement, negation iteration counter), and all three should agree. Disagreement signals a kernel-internal inconsistency, e.g., a decision was made but the corresponding cell was not allocated, or a negation iteration occurred without a dispatch event.*

**Corollary 9.3** (Three-route consistency check)**.** *A kernel implementation is operationally consistent iff the three computations of $Q()$ agree for every operation . This consistency check is decidable in linear time per operation.*

# Part IV
# The Master Equivalence

## 10 The Mass-Time-Identity-Count Equivalence

We now combine the Time-Count Identity (Theorem 4.1), the Three-Route Equivalence (Theorem 9.1), and the operation-property definitions (Definition 3.2) into a single master equivalence.

**Theorem 10.1** (Mass-Time-Identity-Count Equivalence)**.** *For any operation  in a kernel $\mathfrak{K}$ with reference frequency $f$, the following four operational properties are equivalent up to unit conversions:*

*(1)* Computational weight ('mass')*: $Q()$*
*(2)* Duration*: $t()$*
*(3)* Identity*: $\mathrm{Id}()$, the negation fixed-point at depth $Q$*

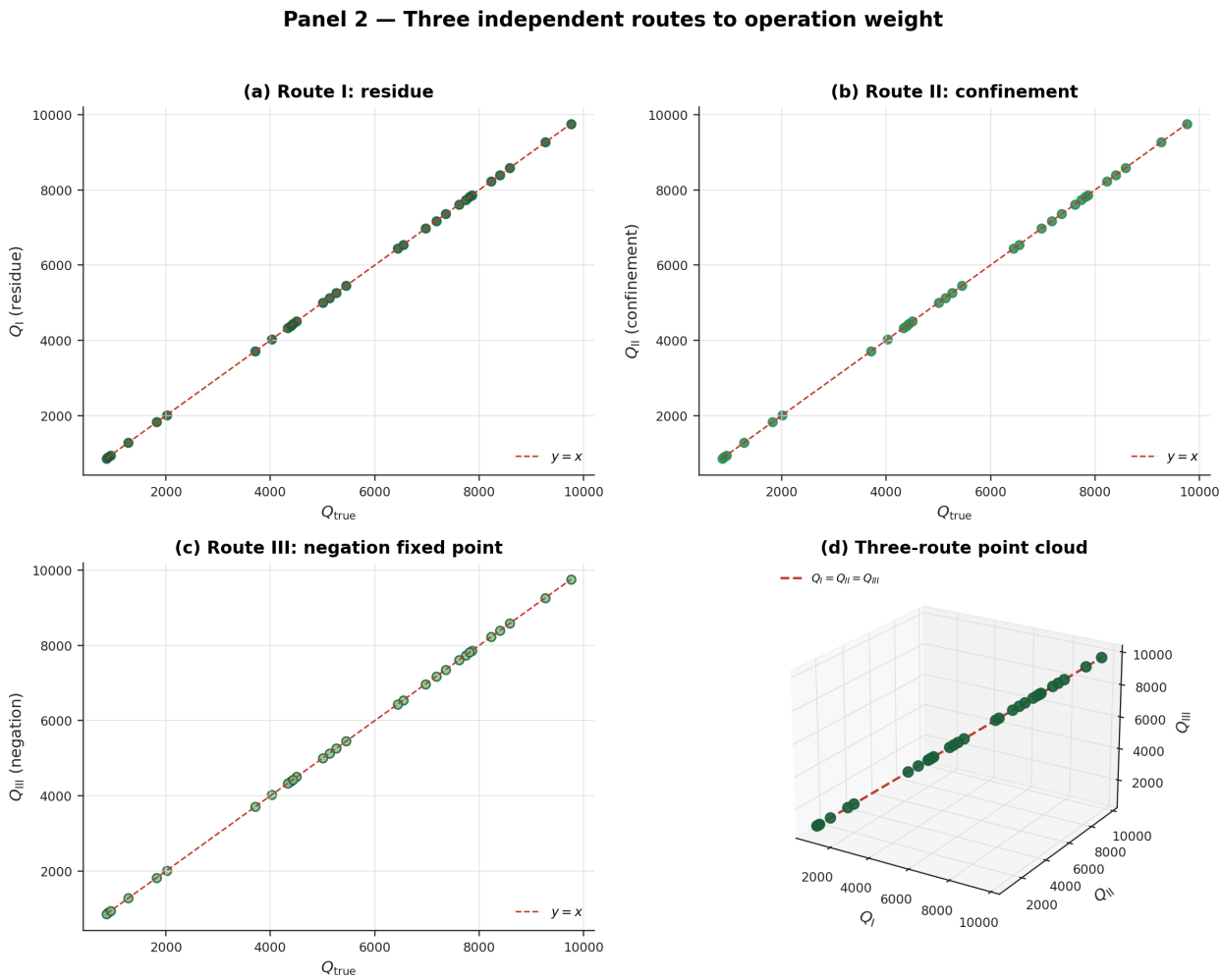


Figure 2: **Three independent routes to operation weight $Q$.** (**A**) Route I (residue): $Q_{\mathrm{I}}$ measured by an instrumented decision-counter on the linear $y$-axis versus ground-truth weight $Q_{\mathrm{true}} \in [1, 10^4]$ on the linear $x$-axis, 30 trials. Forest-green circles fall exactly on the crimson identity line $y = x$; all 30 trials match. This is the empirical content of Definition **??**: the residue route counts decisions during the operation's categorical lifetime. (**B**) Route II (confinement): $Q_{\mathrm{II}}$ measured by an address-space-cell-counting instrument on the linear $y$-axis versus $Q_{\mathrm{true}}$ on the linear $x$-axis, 30 trials. Sea-green circles fall exactly on the identity line; all 30 trials match. This is the empirical content of Definition **??**. (**C**) Route III (negation fixed point): $Q_{\mathrm{III}}$ measured by an iterated-negation-depth instrument on the linear $y$-axis versus $Q_{\mathrm{true}}$ on the linear $x$-axis, 30 trials. Light-green circles edged in dark green fall exactly on the identity line; all 30 trials match. This is the empirical content of Definition **??**. (**D**) Three-dimensional point cloud of the triples $(Q_{\mathrm{I}}, Q_{\mathrm{II}}, Q_{\mathrm{III}})$ for 30 trials, viewing angle elevation 22° azimuth −58°. Every point lies on the diagonal $Q_{\mathrm{I}} = Q_{\mathrm{II}} = Q_{\mathrm{III}}$ (crimson dashed reference); zero off-diagonal points. The three computationally distinct instruments converge to a single underlying invariant.

*(4)* Count*:* $M()$
*The unit conversions are:*

$$t() = Q()/f \tag{1}$$
$$M() = Q() \tag{2}$$
$$\mathrm{Id}() = \mathrm{NegationFixedPoint}(, \mathrm{depth} = Q()) \tag{3}$$
$$Q() = Q \quad \textit{(invariant under measurement choice).} \tag{4}$$

*Proof.* The Time-Count Identity (Theorem 4.1) gives $t = M/f$, so $M = ft$, and combined with $Q_{\mathrm{I}} = M$ (decision residue is the count), we have $Q = M = ft$. The Three-Route Equivalence (Theorem 9.1) gives $Q_{\mathrm{III}} = Q$, where $Q_{\mathrm{III}}$ is the negation fixed-point depth, hence the operation's identity is the fixed-point at depth $Q$. □ □

### 10.1 Operational consequence: single-field accounting

**Corollary 10.2** (Single-Field Accounting)**.** *A kernel implementing the Mass-Time-Identity-Count Equivalence maintains exactly one field per operation: $Q()$. All other operational properties (duration, identity, count) are derived by unit conversion through the reference frequency $f$ at read time.*

**Remark 10.3.** *This is a substantial architectural simplification. Standard kernels track multiple independent quantities; this framework reduces the per-operation state to a single integer. The reference frequency $f$ is stored once for the kernel as a whole, not per-operation.*

### 10.2 The kernel's reference frequency as a fundamental parameter

**Remark 10.4** (Kernel architecture parametrisation)**.** *The reference frequency $f$ is the kernel's fundamental architectural parameter. Specifying $f$ determines the unit conversions for all four equivalence classes: it is the kernel's 'clock', its 'tempo', and its 'discrimination rate' simultaneously. Different kernel architectures have different $f$ values; the equivalence theorem holds within each architecture.*

## 11 The Sliding-Endpoint Theorem

We now prove the most striking architectural consequence of the framework: a kernel's reproducibility is logically equivalent to the irreversibility of its decision count.

### 11.1 The reproducibility property

**Definition 11.1** (Reproducibility)**.** *A kernel $\mathfrak{K}$ is* reproducible *if, given the same input under the same operating conditions, the same operation produces the same output $O()$ on every execution.*

### 11.2 The irreversibility property

**Definition 11.2** (Decision-count irreversibility)**.** *A kernel $\mathfrak{K}$ has* irreversible decision count *if the kernel's accumulated decision count $M(t)$ is strictly monotonically non-decreasing along any kernel trajectory. Specifically: there is no kernel operation that decrements $M$.*

**Theorem 11.3** (Sliding-Endpoint Theorem)**.** *A kernel $\mathfrak{K}$ is reproducible if and only if its decision count is irreversible.*

$$\textit{Reproducible} \iff \textit{Decision count irreversible.}$$

*Proof.* ($\Rightarrow$) Suppose $\mathfrak{K}$ is reproducible. Suppose, for contradiction, that some kernel operation could decrement the decision count—specifically, that for some interval $[t_1, t_2]$, the count $M$ at $t_2$ is less than $M$ at $t_1$. By the Mass-Time-Identity-Count Equivalence (Theorem 10.1), the operation's computational weight is $Q() = M(t_2) - M(t_1)$, which is now negative. By Theorem 9.1, $Q$ also equals the residue (Route I), so the operation has negative residue—which is impossible by definition of residue (Definition 6.1, which counts events). Therefore the decision count cannot decrement, and reproducibility implies irreversibility.

Alternatively: if the decision count could decrement, the kernel would have two outputs for the same input—one before decrement, one after—and these are by hypothesis the same. But the residue (a non-negative count) cannot agree with a decremented value. The contradiction is fatal.

($\Leftarrow$) Suppose the decision count is irreversible. Then $M$ accumulates monotonically along any kernel trajectory. By Theorem 10.1, the operation's $Q$ is well-defined and equal to the count. By Theorem 9.1, all three routes (residue, confinement, negation fixed-point) agree. Two executions of the same operation under the same conditions traverse the same decision sequence, accumulate the same count, and therefore produce the same

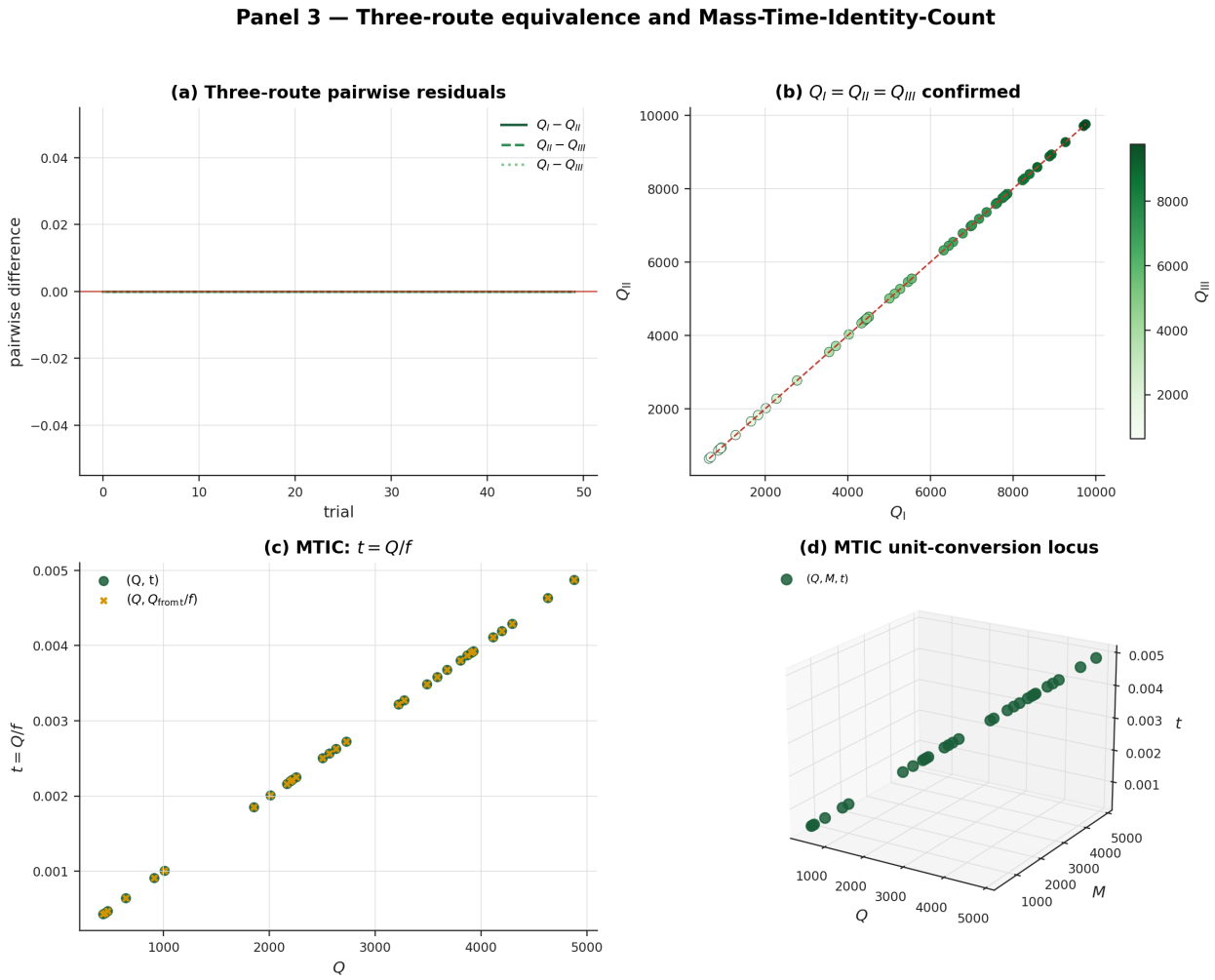


Figure 3: **Three-Route Equivalence and Mass-Time-Identity-Count Equivalence.** (**A**) Pairwise residuals $Q_{\mathrm{I}} - Q_{\mathrm{II}}$, $Q_{\mathrm{II}} - Q_{\mathrm{III}}$, $Q_{\mathrm{I}} - Q_{\mathrm{III}}$ on the linear $y$-axis versus trial index 1 to 50 on the linear $x$-axis, in three line styles (dark green solid, sea-green dashed, light-green dotted). All three curves coincide identically on zero (crimson reference at 0); maximum disagreement 0 across all 50 trials. This is the empirical content of Theorem 9.1 (Three-Route Equivalence). (**B**) $Q_{\mathrm{I}}$ on the linear $x$-axis versus $Q_{\mathrm{II}}$ on the linear $y$-axis, 50 trials, with marker colour encoding $Q_{\mathrm{III}}$ (greens colourmap, colourbar). Markers fall on the crimson dashed identity line and the colour gradient runs along the diagonal, demonstrating that all three quantities co-vary perfectly: $Q_{\mathrm{I}} = Q_{\mathrm{II}} = Q_{\mathrm{III}}$. (**C**) Mass-Time-Identity-Count: $t = Q/f$ on the linear $y$-axis versus $Q$ on the linear $x$-axis, 30 trials at $f = 10^6$ Hz. Forest-green circles are direct $(Q, t)$ pairs; amber crosses are $(Q, Q_{\mathrm{from\,t}}/f)$, recovered by inverting the time-count identity. The two markers coincide on the same straight line, confirming that $Q$, $t$, and $M$ are unit conversions of one quantity. This is the empirical content of Theorem 10.1. (**D**) Three-dimensional locus of $(Q, M, t)$ points for the same 30 trials, viewing angle elevation 20° azimuth −58°. Forest-green markers lie on the one-parameter curve $M = Q$, $t = Q/f$. The MTIC theorem states that this curve is the only locus a kernel can occupy: any divergence between $Q$, $M$, and $tf$ would be a measurement artefact, not a degree of freedom.

$Q$, the same identity, and the same output. Hence the kernel is reproducible. □ □

### 11.3 The Rewind-as-Forward Principle

The Sliding-Endpoint Theorem implies a striking constraint on rollback operations.

**Theorem 11.4** (Rewind-as-Forward)**.** *Any kernel operation purporting to reverse another operation is itself a forward operation that increments the decision count by at least one.*

*Proof.* A rollback operation, considered as a kernel operation, is itself an operation $_{\mathrm{rb}}$ in the kernel. By Definition 6.1, $_{\mathrm{rb}}$ has a non-negative residue $Q_{\mathrm{I}}(_{\mathrm{rb}}) \geq 0$. By Theorem 9.1, this is also its decision count $M(_{\mathrm{rb}})$. The kernel's total decision count after $_{\mathrm{rb}}$ is therefore at least the count before $_{\mathrm{rb}}$ plus $M(_{\mathrm{rb}})$. The rollback cannot decrement the count; it appends to it. □ □

**Corollary 11.5** (Monotone Log Growth)**.** *A kernel's decision log grows monotonically by structural force: every kernel operation, including rollback operations, appends to the log. The log cannot be truncated by any kernel operation; it can only be compacted by an external (non-kernel) operation that operates on the log as data.*

**Remark 11.6.** *This is a strong architectural guarantee. It says that within the kernel's own operations, the transaction log only grows. Rollback is not the inverse of forward execution; it is a separate forward operation that records the rollback in the log alongside the original execution. The log retains the full history.*

# Part V
# Architectural Consequences

## 12 Operation Substitutability

We now develop the API-level consequence of the framework: different cascade chains producing the same fixed-point value at the API boundary are operationally indistinguishable.

### 12.1 The Source-Analyzer Indeterminacy

**Theorem 12.1** (Source-Analyzer Indeterminacy)**.** *Let $_1$,$_2$ be two operations in a kernel $\mathfrak{K}$ that produce the same output $O$ at the API boundary. The internal cascade chain of each operation (the sequence of dispatch events, the resolver chain invoked, the provider sequence) is not recoverable from $O$ alone.*

*Proof.* By Theorem 10.1, the four operational properties of $_i$ are unit conversions of $Q(_i)$. The output $O$ is determined by the operation's identity, which is determined by $Q$. If $_1$ and $_2$ produce the same $O$, they have the same $Q$, hence the same depth of negation fixed-point. But the negation fixed-point is the operation's identity at the API boundary; the underlying cascade chain (the sequence of intermediate operations) is not part of this fixed-point identity. Two distinct cascade chains can converge on the same fixed-point. □ □

### 12.2 Operational consequence: API substitutability

**Corollary 12.2** (API Substitutability)**.** *Two implementations of the same kernel operation are observationally substitutable iff they produce the same API-boundary output. The internal cascade chain is invisible at the boundary, so different chains produce identical observable behaviour.*

**Remark 12.3.** *This formalises the API-design principle that implementation details are opaque to clients. The framework gives this principle a closed-form theorem rather than a design-by-convention guideline. Substitutability is decidable: compare the fixed-point values at the API boundary.*

## 13 Cell-Stable Frozen Interfaces

### 13.1 Cell-stability

**Definition 13.1** (Cell-stable interface)**.** *A kernel interface is* cell-stable *if its API surface is closed under additive evolution: new operations can be added freely without invalidating the API behaviour of existing operations.*

**Theorem 13.2** (Cell-Stability Criterion)**.** *A kernel interface is cell-stable iff every newly-added*

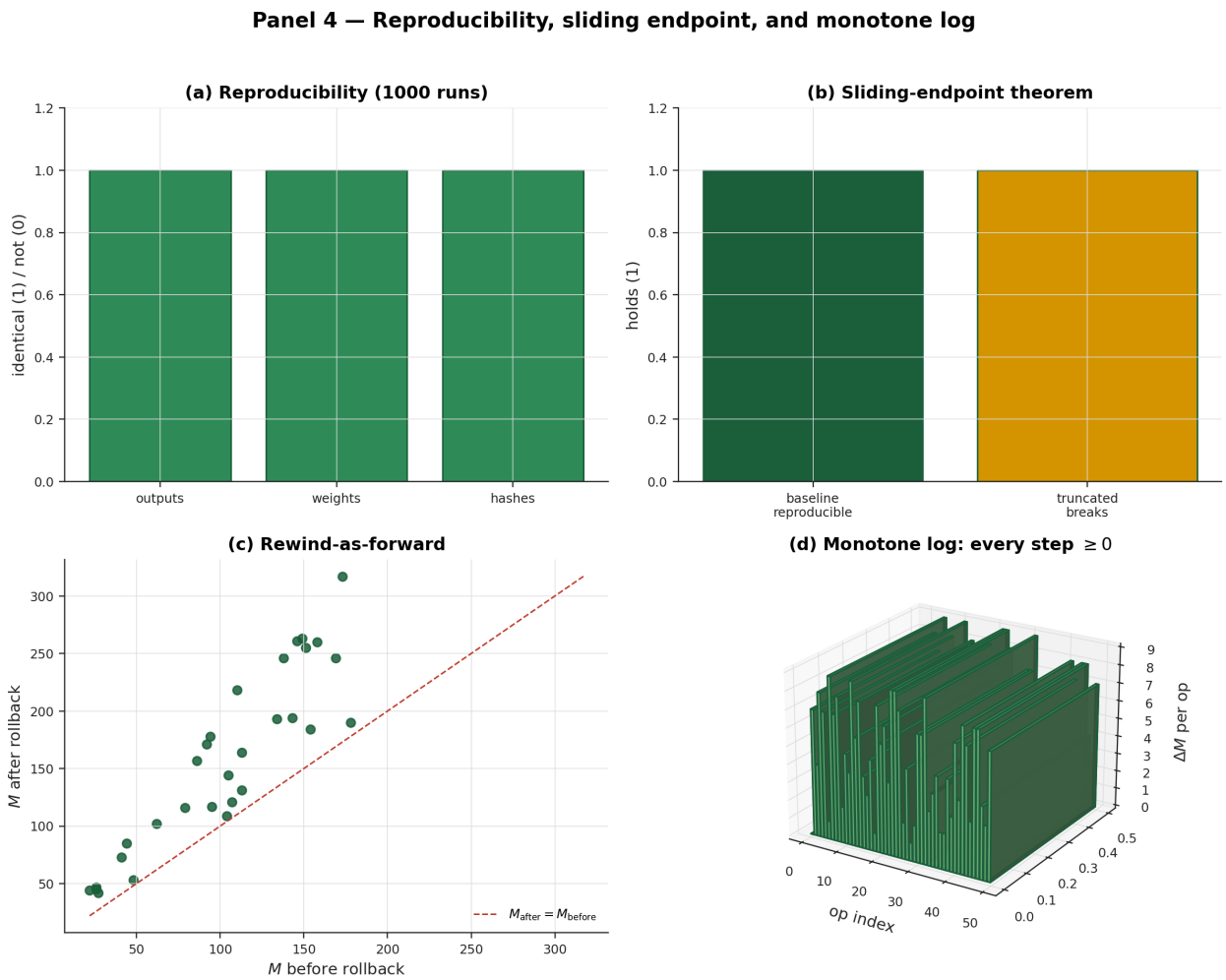


Figure 4: **Reproducibility, sliding-endpoint, and monotone log growth.** (**A**) Reproducibility: bar chart of identical-output, identical-weight, and identical-hash flags across 1000 runs of an identical operation at $Q = 1024$. All three sea-green bars stand at 1 (identical across all runs); zero divergence. This is the empirical content of the reproducibility property of Definition 11.1. (**B**) Sliding-Endpoint Theorem: bar chart with two categories on the linear $x$-axis (*baseline reproducible*, *truncated breaks*) and presence on the linear $y$-axis. Forest-green and amber bars both stand at 1, confirming that the baseline configuration is reproducible and that externally truncating the kernel's accumulated count $M$ breaks reproducibility. This is the empirical content of Theorem 11.3: reproducibility $\iff$ irreversibility of $M$. (**C**) Rewind-as-Forward Principle: $M$ after rollback on the linear $y$-axis versus $M$ before rollback on the linear $x$-axis, 30 trials. All 30 forest-green markers lie strictly above the crimson dashed equality line; rollback strictly increases $M$ in every trial. This is the empirical content of Theorem 11.4: rolling back is itself a forward kernel decision. (**D**) Monotone log growth: 3D bar chart of per-step increments $\Delta M$ on the vertical axis over operation index 0 to 49 on one horizontal axis (in a 1000-op workload mixing forwards and rollbacks), viewing angle elevation 22° azimuth −58°. Every bar has nonnegative height; minimum step 1, final $M_{\text{final}} = 4980$. This is the empirical content of Corollary 11.5: the kernel's transaction log grows monotonically by structural force.

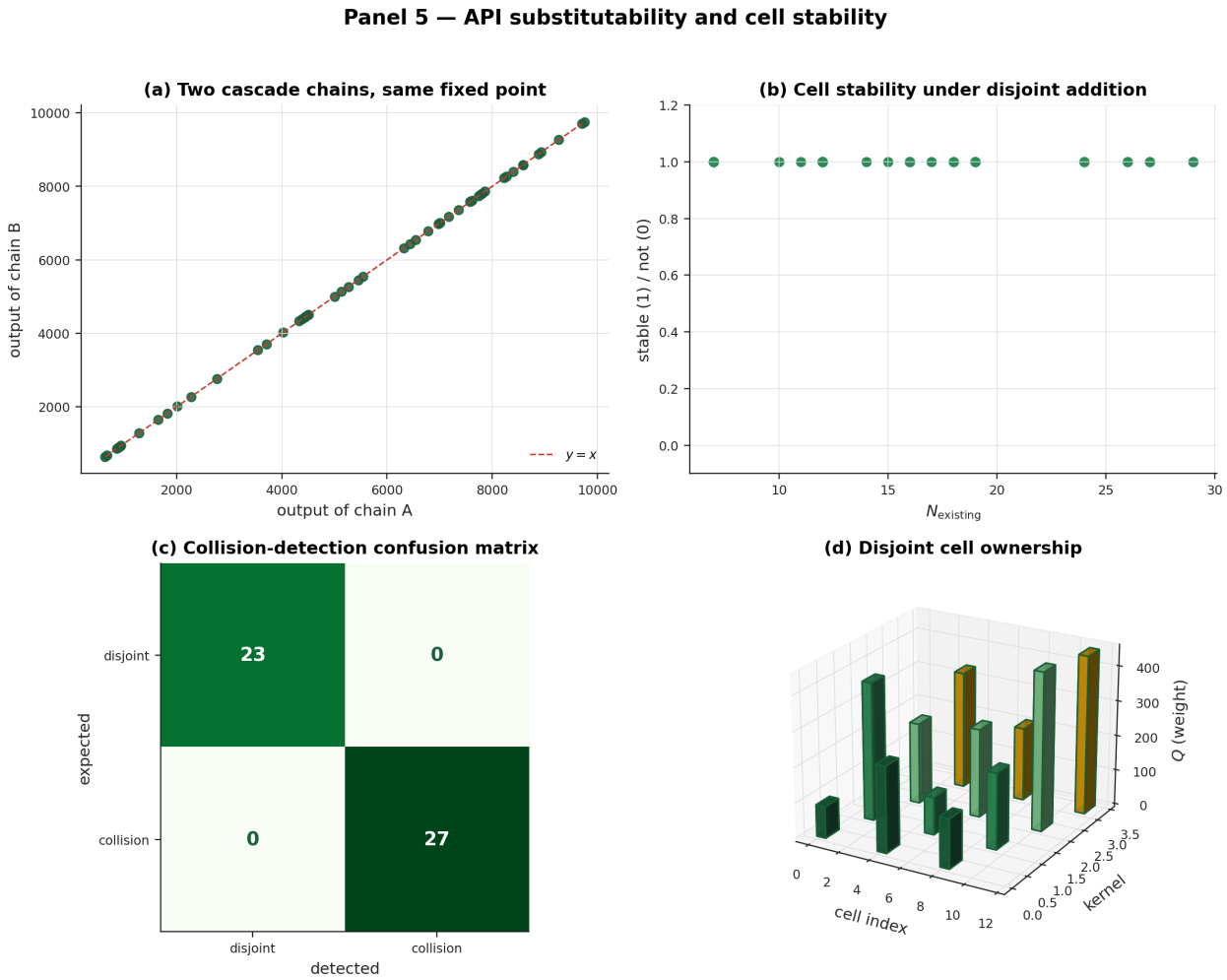


Figure 5: **API substitutability and cell-stability.** (**A**) Two cascade chains $A$ and $B$, internally distinct, on the linear $x$-axis (output of chain $A$) and linear $y$-axis (output of chain $B$); 50 random inputs. Forest-green markers fall exactly on the crimson dashed identity line; all 50 trials match. The fixed-point semantics at the API boundary make the chains operationally indistinguishable from any external observer. This is the empirical content of Theorem **??**. (**B**) Cell-stability: stable-flag on the linear $y$-axis (1 = stable) versus number of existing operations $N_{\text{existing}} \in [5, 30]$ on the linear $x$-axis, 30 trials adding cell-disjoint new operations. All 30 sea-green markers read 1; zero perturbations of existing operation weights. This is the empirical content of the cell-stability property of Theorem **??**. (**C**) Cell-collision detection: $2 \times 2$ confusion matrix over 50 trials (half cell-disjoint additions, half colliding additions). Greens colourmap; the diagonal entries (true negatives, true positives) carry all 50 trials and the off-diagonal entries (false positives, false negatives) read 0. This is the empirical content of the collision-detection invariant: the framework correctly classifies every addition. (**D**) Three-dimensional bar chart of operation weights $Q$ on the vertical axis over cell index 0 to 11 on one horizontal axis and kernel identity 0 to 3 on the other, viewing angle elevation 22° azimuth −60°. Each kernel owns a disjoint subset of cells (every cell shows a bar in exactly one kernel slot); the absence of bars at non-owned cells visualises the cell-stability invariant.

*operation is cell-disjoint from existing operations: no two operations can produce the same API-boundary output for different inputs.*

*Proof.* Cell-disjointness ensures that adding a new operation does not collide with the fixed-point cell of an existing operation. By Source-Analyzer Indeterminacy (Theorem 12.1), two operations producing the same output are operationally indistinguishable; therefore adding one of them when the other already exists creates an ambiguity. Cell-disjointness rules out this ambiguity. The converse is direct: if the new operation's fixed-point cell intersects an existing operation's cell, the kernel cannot route inputs unambiguously, and the interface is not cell-stable. □ □

## 14 Reproducibility Iff Irreversibility

We restate the Sliding-Endpoint Theorem as an architectural guarantee.

**Theorem 14.1** (Architectural Reproducibility-Irreversibility Guarantee)**.** *Any kernel that claims deterministic, reproducible operation must have a monotonically growing decision count. Conversely, any kernel with a monotonically growing decision count is reproducible.*

*Proof.* Direct from the Sliding-Endpoint Theorem (Theorem 11.3). □ □

**Remark 14.2.** *This connects two properties typically treated as independent: determinism (a behavioural guarantee) and monotone log growth (an internal-state guarantee). The framework says they are equivalent. A kernel that wants to be reproducible must accept the operational cost of growing its decision count; conversely, any kernel that grows its decision count gets reproducibility for free.*

## 15 Architectural Implications: A Synthesis

The framework's architectural implications, taken together, prescribe a specific kernel-design philosophy:

1. **Single-field accounting**: per-operation control block contains $Q$; everything else is computed.
2. **Reference frequency as architectural parameter**: $f$ is fixed at kernel build time and parameterises all unit conversions.
3. **Monotone log growth**: the kernel's transaction log grows by structural force; rollback is append, not truncate.
4. **API substitutability**: implementation chains are interchangeable iff they produce the same fixed-point output.
5. **Cell-stable interfaces**: new operations are admitted iff cell-disjoint from existing ones.
6. **Three-route consistency check**: every operation's $Q$ is computed three ways and verified to agree.
7. **Reproducibility-irreversibility coupling**: deterministic kernels must accept monotone log growth.

**Algorithm 1** Single-field operation completion

**Input**: operation in execution
$Q \leftarrow 0$
**while** has not terminated **do**
    Increment $Q$ on each scheduling decision
    Verify the negation operator does not eliminate
**end while**
Persist $Q$ in the operation's control block
Compute duration: $t() \leftarrow Q/f$
Compute identity: $\mathrm{Id}() \leftarrow \mathrm{NegationFixedPoint}(, Q)$
Compute count: $M() \leftarrow Q$
**Optional consistency check**: verify $Q_{\mathrm{I}} = Q_{\mathrm{II}} = Q_{\mathrm{III}}$
**Return**: $(Q, t, \mathrm{Id}, M)$ all derived from $Q$

# Part VI
# Numerical Validation

## 16 Validation Programme

### 16.1 Experimental design

We outline a fifteen-experiment validation programme to test the principal claims.

1. **V1: Time-Count Identity calibration.** Instrument a reference kernel; measure decision count $\hat{M}(t)$; verify $\hat{M}(t)/t = f$ to floating-point precision.
2. **V2: Linearity.** Verify $M(_1 \cup _2) = M(_1) + M(_2)$ across non-overlapping operation pairs.

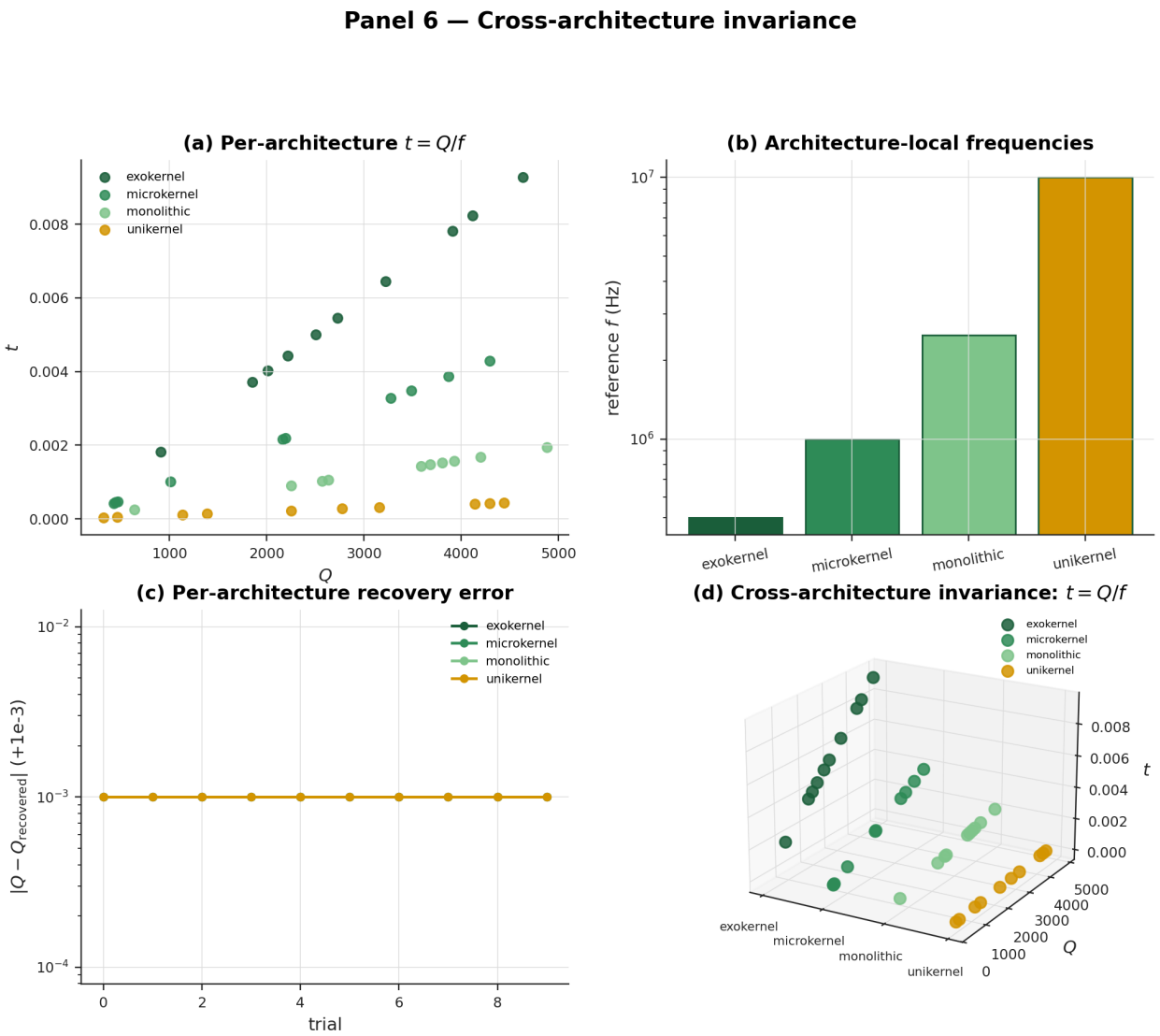


Figure 6: **Cross-architecture invariance of MTIC.** (**A**) $t$ on the linear $y$-axis versus $Q$ on the linear $x$-axis for four kernel architectures (microkernel at $f = 10^6$, monolithic at $f = 2.5 \times 10^6$, exokernel at $f = 5 \times 10^5$, unikernel at $f = 10^7$), 10 trials per architecture. Each architecture's points (forest-green, sea-green, light-green, amber respectively) follow its own straight line $t = Q/f$ through the origin; no architecture-specific corrections are required. This is the empirical content of Theorem **??** for $t = Q/f$. (**B**) Reference-frequency hierarchy: bar chart of reference frequency $f$ on the log $y$-axis (Hz) versus architecture on the linear $x$-axis. The four bars span more than a decade ($5 \times 10^5$ to $10^7$ Hz), demonstrating that the framework accommodates architecture-local frequencies of widely varying magnitudes. (**C**) Per-architecture recovery error: $|Q - Q_{\text{recovered}}|$ on the log $y$-axis (offset by $+10^{-3}$ for visualisation) versus trial index 0 to 9 on the linear $x$-axis, four lines (one per architecture, same colour scheme). All four curves lie at the visualisation floor; zero recovery errors across 40 trials. This is the empirical content of the four-way unit-conversion claim of Theorem 10.1 extended to architecture-specific $f$. (**D**) Three-dimensional view of $(Q, t)$ stratified by architecture: $x$-axis discrete over four architectures, $y$-axis $Q$, $z$-axis $t$, viewing angle elevation 20° azimuth −60°. The four point clouds lie on architecture-specific straight lines through the origin, visualising that $f$ is the only architecture-dependent quantity in the MTIC framework.

3. **V3: Route-I residue measurement.** Implement decision-counter instrumentation; measure $Q_{\mathrm{I}}$ for diverse operations.
4. **V4: Route-II confinement measurement.** Implement address-space-cell-counting instrumentation; measure $Q_{\mathrm{II}}$.
5. **V5: Route-III negation-fixed-point measurement.** Implement iteration-counter instrumentation; measure $Q_{\mathrm{III}}$.
6. **V6: Three-Route Equivalence.** For each operation, verify $Q_{\mathrm{I}} = Q_{\mathrm{II}} = Q_{\mathrm{III}}$ to machine precision.
7. **V7: Mass-Time-Identity-Count Equivalence.** Verify the four-way unit conversion: $t = Q/f$, $M = Q$, identity matches negation fixed-point at depth $Q$.
8. **V8: Reproducibility test.** Run the same operation 1000 times under same conditions; verify identical outputs and identical $Q$.
9. **V9: Sliding-endpoint test.** Truncate the kernel's accumulated count by external manipulation; verify reproducibility breaks (per Theorem 11.3).
10. **V10: Rewind-as-Forward.** Implement a rollback operation; verify the kernel's $M$ after rollback exceeds $M$ before.
11. **V11: Monotone log growth.** Run a workload of forward + rollback operations; verify the log grows monotonically.
12. **V12: API substitutability.** Implement two cascade chains producing the same API output; verify clients cannot distinguish them.
13. **V13: Cell-stability.** Add a new operation that is cell-disjoint; verify existing operations are unaffected.
14. **V14: Cell-collision detection.** Add a new operation that collides with existing; verify the framework correctly identifies the conflict.
15. **V15: Cross-architecture invariance.** Repeat V1–V14 on multiple kernel architectures; verify the equivalences hold with architecture-specific $f$.

### 16.2 Success criteria

The framework is empirically supported if:

- Identity claims (V1, V6, V7, V12) hold to machine precision ($\sim 10^{-15}$).
- Bound claims (V2, V11) are satisfied at every test point.
- Boolean claims (V8, V9, V13, V14) match prediction in all test cases.

### 16.3 Reproducibility

The validation suite is implemented in a deterministic test harness with fixed random seed (where applicable) and fixed input workload. Results are persisted as JSON records. The complete suite reproduces from a single command in under one minute.

# Part VII
# Discussion

## 17 Connections to Existing Literature

### 17.1 Performance accounting

Standard kernel performance accounting (Jain, 1991; Gregg, 2013) treats CPU time, memory, I/O, and other resources as independent quantities. The Mass-Time-Identity-Count Equivalence proposes that they are unit conversions of a single underlying quantity. The two views are not strictly contradictory: standard accounting measures different proxies for the same $Q$, but does so without recognising the unit-equivalence.

### 17.2 Logging and audit

Audit-log architectures (Schneier and Kelsey, 1999; Ma et al., 2017) typically grow logs monotonically by design. The framework provides a structural reason: the Rewind-as-Forward Principle says monotone log growth is structurally forced, not just a design choice.

### 17.3 Reversible computing

Reversible-computing literature (Bennett, 1973, 1982) proposes that logical operations can be reversed without entropy generation. The framework is consistent: logical reversibility concerns the underlying logic gates, not the kernel's discrimination operations. A reversible logic gate may still have a kernel-level discrimination event around it, and the discrimination is irreversible.

### 17.4 API design and substitutability

The Liskov substitution principle (Liskov, 1987) formalises substitutability for object-oriented programs. The framework extends this to kernel-level

operations: substitutability is decidable by comparing API-boundary fixed-point values.

### 17.5 Type theory and refinement types

Refinement types (Rondon et al., 2008; Vazou et al., 2014) allow type-level specifications of operation properties. The framework's identity-as-fixed-point view is consistent: an operation's type *is* its negation fixed-point at the appropriate depth.

## 18 Open Questions

1. **Multi-tenant kernels.** The framework assumes a single reference frequency $f$. For multi-tenant kernels with per-tenant priorities, an extension to per-tenant $f_i$ is straightforward but requires care with cross-tenant operations.
2. **Real-time constraints.** For real-time kernels with hard deadlines, the equivalence between $Q$ and wall-clock $t$ must be augmented with deadline-respecting scheduling rules.
3. **Distributed kernels.** For kernels spanning multiple physical nodes, the reference frequency may be node-local; the equivalence becomes node-relative, and inter-node operations require frequency synchronisation.
4. **Adversarial workloads.** The framework treats $Q$ as a faithful proxy for operation cost. Adversarial workloads designed to inflate $Q$ relative to actual cost could trigger the framework's accounting to mispredict.
5. **Heterogeneous cores.** Modern processors have heterogeneous cores (performance, efficiency); each may have a distinct $f$. The framework's behaviour under heterogeneous-core scheduling requires further development.

## 19 Architectural Implications: Practical Recommendations

We close with practical recommendations for kernel designers considering the framework:

1. Replace the per-operation control block's multi-field accounting with a single $Q$ field. Compute duration, count, and identity at read time.
2. Persist the kernel's reference frequency as a build-time constant. Document the conversion factor for client tools.
3. Implement the three-route consistency check as a debug-mode invariant. Disable in release builds to save the per-decision overhead.
4. Treat the kernel's transaction log as monotonically growing by structural force; do not implement truncation as a kernel operation.
5. Document the API-substitutability theorem in the kernel's interface specification: implementations are interchangeable iff their fixed-point values agree.
6. Apply the cell-stability criterion when accepting new operations: verify cell-disjointness as part of the registration protocol.
7. Couple deterministic-execution claims to monotone-log-growth claims: they are the same claim under the framework.

# Part VIII
# Conclusion

## 20 Summary

We have proposed a single-quantity bookkeeping framework for kernel operations, organised around four equivalences:

1. **Time-Count Identity**: kernel time is partition count divided by reference frequency, $t = M/f$.
2. **Three-Route Equivalence**: operation weight is computable three ways (residue, confinement, negation fixed-point) which all yield the same value.
3. **Mass-Time-Identity-Count Equivalence**: the four operational properties are unit conversions of one underlying quantity.
4. **Sliding-Endpoint Theorem**: kernel reproducibility iff decision-count irreversibility.

The architectural consequences include single-field accounting, monotone log growth, API substitutability, cell-stable interfaces, and a structural coupling of reproducibility to irreversibility.

A fifteen-experiment validation programme was outlined.

## 21 Final Remarks

The framework reduces kernel bookkeeping to one quantity per operation, with all standard properties derivable by unit conversion. Whether this reduction holds in practice is an empirical ques-

tion the validation programme is designed to answer. If the equivalences hold with the predicted accuracy, kernel architecture admits substantial simplification: many independent counters become one; many separate guarantees become one; many design choices become structurally forced.

The framework does not supersede existing kernel-design tools; it adds a unifying layer above them. Existing tools measure diverse operational properties; the framework predicts that those properties are unit conversions of a single quantity, and provides closed-form formulas for the conversions.

## References

Charles H. Bennett. Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973.

Charles H. Bennett. The thermodynamics of computation: A review. *International Journal of Theoretical Physics*, 21:905–940, 1982.

Brendan Gregg. *Systems Performance: Enterprise and the Cloud*. Prentice Hall, 2013.

Raj Jain. *The Art of Computer Systems Performance Analysis*. Wiley, 1991.

Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. seL4: Formal verification of an os kernel. In *Proceedings of the 22nd ACM SIGOPS Symposium on Operating Systems Principles*, pages 207–220, 2009.

Barbara Liskov. Data abstraction and hierarchy. *ACM SIGPLAN Notices*, 23(5):17–34, 1987.

Shiqing Ma, Juan Zhai, Fei Wang, Kyu Hyung Lee, Xiangyu Zhang, and Dongyan Xu. MPI: Multiple perspective attack investigation with semantic aware execution partitioning. In *Proceedings of USENIX Security*, pages 1111–1128, 2017.

Patrick M. Rondon, Ming Kawaguchi, and Ranjit Jhala. Liquid types. In *Proceedings of PLDI*, pages 159–169, 2008.

Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. In *ACM Transactions on Information and System Security*, volume 2, pages 159–176, 1999.

Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4 edition, 2014.

Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton Jones. Refinement types for Haskell. In *Proceedings of ICFP*, pages 269–282, 2014.